

Ejercicios para casa

Cómo trabajar este documento

22 ejercicios cortos, organizados por epígrafe, para practicar por tu cuenta lo visto en el Bloque 1. No es necesario hacerlos todos del tirón: puedes ir resolviéndolos a medida que avanzamos en clase con cada epígrafe.

- Un archivo .py por ejercicio, dentro de una misma carpeta ejercicios_bloque1 con su propio entorno virtual (ver Ejercicio 0.1).
- Nombra los archivos de forma reconocible, por ejemplo ej04_adivina_el_tipo.py.
- Todas las variables, declaradas, inicializadas y anotadas con su tipo — la misma regla que en la Entrega 0.
- Nada de bucles (for, while) ni condicionales (if): todavía no los hemos visto. Todo se resuelve con variables, tipos, operadores, E/S, casting y comentarios.
- Caracteres siempre ASCII (nada de emojis ni símbolos Unicode en las cadenas).
- Comprueba tú mismo el resultado ejecutando el programa y revisando que VS Code no marque ningún warning.

💡 Este documento no incluye las soluciones a propósito: la idea es que compruebes tú mismo si el resultado es el esperado, ejecutando el código. Si prefieres tener también un documento de soluciones para autocorregirte, dilo y se prepara aparte.

0 · Antes de empezar: tu entorno de trabajo

EJERCICIO 0.1 · Prepara tu carpeta de trabajo [Fácil]

Crea una carpeta llamada ejercicios_bloque1. Dentro, crea un entorno virtual (`python -m venv venv`) y actívalo. Comprueba con `python --version` que lo estás usando (VS Code debe mostrar el intérprete de venv abajo a la derecha).

EJERCICIO 0.2 · Tu primer repositorio [Fácil]

Dentro de esa misma carpeta, ejecuta `git init` para convertirla en un repositorio. Crea un archivo `README.md` con una línea describiendo la carpeta, y haz tu primer commit (`git add .` && `git commit -m "Primer commit: preparo la carpeta de ejercicios"`). Si tienes cuenta de GitHub, súbelo con `git push` como práctica extra.

EJERCICIO 0.3 · Practica con el depurador [Fácil]

Abre cualquiera de tus ejercicios ya resueltos en VS Code. Pon un breakpoint en una línea intermedia (clic junto al número de línea), inicia la depuración con `F5`, y usa `Step Over` para avanzar línea a línea observando cómo cambian las variables en el panel lateral.

💡 Puedes ir haciendo un commit por cada ejercicio que termines: es la mejor forma de acostumbrarte a usar git desde ya, sin esperar a la Entrega 0.

1.2 · Variables, identificadores y PEP 8

EJERCICIO 1 • Ficha de personaje [Fácil]

Declara, inicializa y anota cuatro variables para una ficha de personaje de rol: **nombre (str)**, **nivel (int)**, **vida (float)** y **esta_vivo (bool)**. Imprímelas todas en una sola línea con `print()`.

EJERCICIO 2 • El detective de nombres [Fácil]

Sin escribir código ejecutable: en un archivo de comentarios, indica para cada identificador si es válido o no, y por qué: `3profesor`, `nota_final`, `__init`, `for`, `MediaClase`, `nota-parcial`, `Aprobado2024`.

EJERCICIO 3 • Refactoriza este desastre [Medio]

Reescribe este fragmento con nombres descriptivos, anotación de tipo y PEP 8:

```
x = 1
y = "Ana"
z = True
print(x, y, z)
```

1.3 • Tipos de datos

EJERCICIO 4 • Adivina el tipo [Fácil]

Antes de ejecutar nada, escribe en un comentario qué tipo crees que devuelve cada expresión. Luego compruébalo con `type()`:

```
type(3 + 2)
type(3 / 2)
type(3 // 2)
type("3" + "2")
type(3 != 2)
type("3.5")
```

EJERCICIO 5 • Trocea tu nombre [Fácil]

Declara una variable `str` con tu nombre. Imprime, por separado: la primera letra, la última letra, la longitud total y las tres primeras letras (usando slicing, `[0:3]`).

EJERCICIO 6 • Tamaños con `sys.getsizeof` [Medio — opcional]

Importa `sys` y compara, imprimiéndolos, el tamaño en bytes de un `int` pequeño (por ejemplo 5), un `int` enorme (`10 ** 50`) y una cadena larga (30 caracteres o más). Comenta en una línea qué observas.

1.4 • Operadores

EJERCICIO 7 • Calculadora de vuelta [Fácil]

Declara `precio_producto: float` y `precio_pagado: float` con valores de tu elección (el segundo mayor que el primero). Calcula y muestra el cambio a devolver.

EJERCICIO 8 • División entera con trampa [Fácil]

Declara `total_minutos: int` (por ejemplo 135) y calcula, con `//` y `%`, las horas y los minutos restantes. Repite el cálculo con un valor negativo y explica en un comentario por qué el resultado es el que es (repasa el apartado 1.4 de los apuntes si hace falta).

EJERCICIO 9 • ¿Está en rango? [Medio]

Declara `numero: int` y una variable `bool` llamada `en_rango` que compruebe, con `and`, si `numero` está entre 1 y 100 (ambos incluidos). Pruébalo con un valor dentro y otro fuera del rango (cambiando el valor a mano, no con `input`).

EJERCICIO 10 • Par o impar [Fácil]

Declara `numero: int` con un valor de tu elección y una variable `bool` llamada `es_par` que sea `True` si el número es par, usando el operador `%`. Pruébalo con un valor par y otro impar (cambiándolo a mano).

1.5 • Entrada / salida

EJERCICIO 11 • Tarjeta de presentación [Fácil]

Pide por teclado el nombre, la edad y la ciudad del usuario. Imprime una frase con f-string que combine los tres datos, por ejemplo:

```
"Hola, soy Ana, tengo 20 años y vivo en Elche."
```

EJERCICIO 12 • Dibuja un marco [Medio]

Sin bucles: usando varios `print(..., end="")` seguidos, dibuja una línea decorativa de al menos 5 repeticiones, por ejemplo `*-*-*-*`. Termina con un `print()` vacío para saltar de línea.

EJERCICIO 13 • Resumen de la compra [Fácil]

Pide tres precios por teclado (`input()`), conviértelos a `float`, calcula el total y muéstralo con una f-string bien formateada, por ejemplo `"Total: 24.50 euros"`.

1.6 • Conversión de tipos (casting)

EJERCICIO 14 • Conversor de temperatura [Fácil]

Pide una temperatura en grados Celsius por teclado, conviértela a `float`, calcula su equivalente en Fahrenheit (`celsius * 9 / 5 + 32`) y muestra el resultado con una f-string.

EJERCICIO 15 • Arregla la suma [Fácil — depuración]

Este fragmento no suma los dos números, los concatena como texto. Corrígelo con casting:

```
a: str = input("Primer número: ")
b: str = input("Segundo número: ")
print(a + b)
```

EJERCICIO 16 • Trunca o redondea [Medio]

Pide un número con decimales por teclado, conviértelo primero a `float` y después a `int`. Muestra el resultado y explica en un comentario si `int()` ha redondeado o truncado el valor.

1.7 · Comentarios y documentación

EJERCICIO 17 · Documenta tu mejor ejercicio [Fácil]

Elige uno de los ejercicios anteriores que te haya costado más y añádele un docstring inicial explicando qué hace el programa, más al menos un comentario que aporte contexto real (no que repita lo que ya dice el código).

EJERCICIO 18 · Traduce un mal comentario [Fácil]

Este comentario solo repite lo que ya dice el código. Reescríbelo explicando el porqué de esa línea, en el contexto de un tablero de ajedrez:

```
fila: int = fila + 1 # sumamos uno a fila
```

Reto integrador

EJERCICIO 19 · Calculadora de propina [Reto]

Combina todo lo del bloque: pide el importe de una cuenta y el porcentaje de propina deseado, calcula el importe de la propina y el total a pagar, y muéstralo todo con f-strings bien formateadas. Variables anotadas, comentarios donde aporten, sin bucles ni condicionales.

 Este reto es un buen calentamiento para la Entrega 0: usa exactamente las mismas herramientas (variables, tipos, operadores, E/S, casting, comentarios) sobre un problema distinto al del tablero.